Mohammed Abdul Rahim

Classification – Model Selection

Lab Task - 1

Predict the given sample is class=2(malignant) or class=4(tumor) breast cancer (dependent variable) by means of classification model using the following parameters (independent variables):

1. Clump Thickness
2. Uniformity of Cell Size
3. Uniformity of Cell Shape
4. Marginal Adhesion
5. Single Epithelial Cell Size
6. Bare Nuclei
7. Bland Chromatin
8. Normal Nucleoli
9. Mitoses

All the classification models, such as Logistic, KNN, Naïve Bayes and decision tree should be applied to the dataset, and the best fit model should be chosen for the given dataset.

[**You have to create separate file for each model**]

Note: You have to use BreastCancer.csv

Clump Thickness	Uniformity of Cell Size	Uniformity of Cell Shape	Marginal Adhesion	Single Epithelial Cell Size	Bare Nuclei	Bland Chromatin	Normal Nucleoli	Mitoses	Class
5	1	1	1	2	1	3	1	1	2
5	4	4	5	7	10	3	2	1	2
3	1	1	1	2	2	3	1	1	2
6	8	8	1	3	4	3	7	1	2
4	1	1	3	2	1	3	1	1	2
8	10	10	8	7	10	9	7	1	4
1	1	1	1	2	10	3	1	1	2
2	1	2	1	2	1	3	1	1	2
2	1	1	1	2	1	1	1	5	2

Steps:

1. Import libraries and datasets
2. Split the dataset
3. Feature Scaling
4. Apply model on the Training set
5. Evaluate the model's performance using the confusion matrix and accuracy rate.

Answer:

Logistic Regression:

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset
	`dataset = pd.read_csv('BreastCancer.csv')` `X = dataset.iloc[:, :-1].values`

	`y = dataset.iloc[:, -1].values`
	`print(X)`
	`print(y)`

c.	Encode the categorical dependent variable

	`from sklearn.preprocessing import LabelEncoder`
	`le = LabelEncoder()`
	`y = le.fit_transform(y)`

d.	Splitting the dataset into the Training set and Test set
	`from sklearn.model_selection import train_test_split` `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)`
e.	Feature Scaling

	`from sklearn.preprocessing import StandardScaler`
	`sc = StandardScaler()`
	`X_train = sc.fit_transform(X_train)`
	`X_test = sc.transform(X_test)`

f.	Training the Logistic Regression model on the Training set

	`from sklearn.linear_model import LogisticRegression`
	`classifier = LogisticRegression(random_state = 0)`
	`classifier.fit(X_train, y_train)`

g.	Making the Confusion Matrix

	`from sklearn.metrics import confusion_matrix, accuracy_score`
	`y_pred = classifier.predict(X_test)`
	`cm = confusion_matrix(y_test, y_pred)`
	`print(cm)`
	`accuracy_score(y_test, y_pred)`
	
	`[[103  4]`
	`[ 5  59]]`
	`0.9473684210526315`
	
	Note: Therefore, for the given dataset, 9 errors (4+5) are found and accuracy of the model is 94.73%

h.	Predicting the value

	`print(classifier.predict(sc.transform([[1,1,1,1,2,5,1,1,1]])))`
	`[2]`
	Note: It predicts as class=2(malignant)

K-Nearest Neighbors (K-NN):

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('BreastCancer.csv')`
	`X = dataset.iloc[:, :-1].values`
	`y = dataset.iloc[:, -1].values`
	`print(X)`
	`print(y)`

c.	Splitting the dataset into the Training set and Test set
	`from sklearn.model_selection import train_test_split` `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)`
d.	Feature Scaling

	`from sklearn.preprocessing import StandardScaler`
	`sc = StandardScaler()`
	`X_train = sc.fit_transform(X_train)`
	`X_test = sc.transform(X_test)`

e.	Training the K-NN model on the Training set

	`from sklearn.neighbors import KNeighborsClassifier`
	`classifier = KNeighborsClassifier(n_neighbors = 5, metric = 'minkowski', p = 2)`
	`classifier.fit(X_train, y_train)`

f.	Making the Confusion Matrix

	`from sklearn.metrics import confusion_matrix, accuracy_score`
	`y_pred = classifier.predict(X_test)`
	`cm = confusion_matrix(y_test, y_pred)`
	`print(cm)`
	`accuracy_score(y_test, y_pred)`
	
	`[[103  4]`
	`[ 5  59]]`
	`0.9473684210526315`
	
	Note: Therefore, for the given dataset, 9 errors (4+5) are found and accuracy of the model is 94.73%

g.	Predicting the value
	`print(classifier.predict(sc.transform([[1,1,1,1,2,5,1,1,1]])))` Note: write all the independent variables. The output should show correct class.

Mohammed Abdul Rahim

Naïve Bayes(NB):

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('BreastCancer.csv')`
	`X = dataset.iloc[:, :-1].values`
	`y = dataset.iloc[:, -1].values`
	`print(X)`
	`print(y)`

c.	Splitting the dataset into the Training set and Test set
	`from sklearn.model_selection import train_test_split` `X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.25, random_state = 0)`
d.	Feature Scaling

	`from sklearn.preprocessing import StandardScaler`
	`sc = StandardScaler()`
	`X_train = sc.fit_transform(X_train)`
	`X_test = sc.transform(X_test)`

e.	Training the NB model on the Training set

	`from sklearn.naive_bayes import GaussianNB`
	`classifier = GaussianNB()`
	`classifier.fit(X_train, y_train)`

f.	Making the Confusion Matrix

	`from sklearn.metrics import confusion_matrix, accuracy_score`
	`y_pred = classifier.predict(X_test)`
	`cm = confusion_matrix(y_test, y_pred)`
	`print(cm)`
	`accuracy_score(y_test, y_pred)`
	
	`[[99  8]`
	`[ 2 62]]`
	`0.9415204678362573`
	
	Note: Therefore, for the given dataset, 10 errors (8+2) are found and accuracy of the model is 94.15%

g.	Predicting the value
	`print(classifier.predict(sc.transform([[1,1,1,1,2,5,1,1,1]])))`

Mohammed Abdul Rahim

Decision Tree(DT):

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('BreastCancer.csv')`
	`X = dataset.iloc[:, :-1].values`
	`y = dataset.iloc[:, -1].values`
	`print(X)`
	`print(y)`

c.	Splitting the dataset into the Training set and Test set

	`from sklearn.model_selection import train_test_split`
	`X_train, X_test, y_train, y_test = train_test_split(X, y,`
	`test_size = 0.25, random_state = 0)`

d.	Feature Scaling

	`from sklearn.preprocessing import StandardScaler`
	`sc = StandardScaler()`
	`X_train = sc.fit_transform(X_train)`
	`X_test = sc.transform(X_test)`

e.	Training the DT model on the Training set

	`from sklearn.tree import DecisionTreeClassifier`
	`classifier = DecisionTreeClassifier(criterion = 'entropy',`
	`random_state = 0)`
	`classifier.fit(X_train, y_train)`

f.	Making the Confusion Matrix

	`from sklearn.metrics import confusion_matrix, accuracy_score`
	`y_pred = classifier.predict(X_test)`
	`cm = confusion_matrix(y_test, y_pred)`
	`print(cm)`
	`accuracy_score(y_test, y_pred)`
	
	`[[103  4]`
	`[ 3  61]]`
	`0.9590643274853801`
	
	Note: Therefore, for the given dataset, 7 errors (4+3) are found and accuracy of the model is 95.90%

g.	Predicting the value

```
print(classifier.predict(sc.transform([[1,1,1,1,
2,5,1,1,1]])))
```

From the above,

Model	Errors	Accuracy
LR	9	94.73%
KNN	9	94.73%
NB	10	94.15%
DT	7	95.90%

Thus, Decision Tree model is best fit model for the given dataset (BreastCancer).

Do it Yourself- 1:

Develop and evaluate classification models to predict student pass/fail status based on their performance in 'Quiz', 'Practical', and 'Project' exams. A student is considered 'Pass' if their 'Total' marks exceed 60. Implement and compare classification models such as Logistic Regression, K-Nearest Neighbors (KNN), Support Vector Machines (SVM), Decision Trees, and Random Forest. Evaluate model performance using metrics like accuracy, precision, recall, and F1-score. Select the best-performing model based on these metrics. Print the evaluation metrics for each model and determine the number of students predicted to pass and fail by each model. If the initial model performance is unsatisfactory, consider adjusting the dataset to improve classification accuracy.

Steps to be followed:

Problem Definition

- Define the target variable as a categorical value based on a criterion. For instance:
  - Categorize `Total` into classes such as "Pass" or "Fail" based on a threshold (e.g., `Total>60`).
  - Alternatively, you can create multiple classes (e.g., "Low", "Medium", "High") based on percentiles.

2. Preprocessing the Dataset

- Handle missing values (replace with the mean or median).
- Remove outliers (if necessary).
- Normalize or scale the independent variables (`Quiz`, `Practical`, `Project`, `Final`) if required.
- Create the categorical target variable.

3. Splitting the Dataset

- Split the data into training and testing sets.

4. Applying Classification Models

- Train multiple classification models like:
  - Logistic Regression
  - Decision Tree Classifier
  - Random Forest Classifier
  - Support Vector Machine (SVM)
  - k-Nearest Neighbors (k-NN)
  - Gradient Boosting Classifier
- Evaluate models using metrics like:
  - Accuracy
  - Precision
  - Recall
  - F1-score

- Confusion Matrix

Sample Code:

```
# Importing necessary libraries
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score,
precision_score, recall_score, f1_score
from imblearn.over_sampling import SMOTE

# Load the dataset
file_path = 'Marks.csv'  # Update the file path
data = pd.read_csv(file_path)

# Preprocessing
data = data[['Quiz', 'Practical', 'Project', 'Final']].copy()
data = data.fillna(data.mean())

# Create the target variable 'Total'
data['Total'] = (
   0.1 * data['Quiz'] +
   0.3 * data['Practical'] +
   0.3 * data['Project'] +
   0.3 * data['Final']
)

# Adjust the threshold to 60
threshold = 60
data['Target'] = np.where(data['Total'] >= threshold, 1, 0)

# Print the number of students passed and failed
print("Number of Students Passed:", (data['Target'] == 1).sum())
print("Number of Students Failed:", (data['Target'] == 0).sum())

# Define features (X) and target (y)
X = data[['Quiz', 'Practical', 'Project', 'Final']]
y = data['Target']

# Split the dataset
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

# Apply SMOTE to handle class imbalance
smote = SMOTE(random_state=1)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)

# Models to train
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Gradient Boosting": GradientBoostingClassifier(),
    "SVM": SVC(probability=True)
}

# Evaluate each model
evaluation_metrics = []

for name, model in models.items():
    model.fit(X_train_resampled, y_train_resampled)
    y_pred = model.predict(X_test)

    # Calculate metrics
    accuracy = accuracy_score(y_test, y_pred)
    precision = precision_score(y_test, y_pred, zero_division=0)
    recall = recall_score(y_test, y_pred)
    f1 = f1_score(y_test, y_pred)

    # Store the metrics
    evaluation_metrics.append({
        "Model": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-Score": f1
    })

# Convert metrics to DataFrame
metrics_df = pd.DataFrame(evaluation_metrics)

# Print evaluation metrics for all models
print("\nEvaluation Metrics for All Models:")
print(metrics_df)

# Identify the best-fit model based on F1-Score
best_model = metrics_df.loc[metrics_df['F1-Score'].idxmax()]
print("\nBest-Fit Model:")
```

print(f"Model: {best_model['Model']}")
print(f"Reasons: Highest F1-Score ({best_model['F1-Score']:.2f}), Precision ({best_model['Precision']:.2f}), and Recall ({best_model['Recall']:.2f})")

Expected Output:

```
Number of Students Passed: X
Number of Students Failed: Y

Evaluation Metrics for All Models:
Model                  Accuracy    Precision    Recall    F1-Score
Logistic Regression     0.85        0.83         0.88      0.85
Decision Tree           0.80        0.78         0.83      0.80
Random Forest           0.88        0.90         0.85      0.87
Gradient Boosting       0.87        0.89         0.84      0.86
SVM                     0.82        0.80         0.85      0.82

Best-Fit Model:
Model: Random Forest
Reasons: Highest F1-Score (0.87), Precision (0.90), and Recall (0.85)
```

Do it Yourself- 2:

Try to optmise the above classification models based on breastcancer.csv using cross-validation method.

```
import pandas as pd
from sklearn.model_selection import cross_val_predict, StratifiedKFold
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score

# Load the dataset
data = pd.read_csv('BreastCancer.csv')

# Separate features and target variable
X = data.drop(columns=['Class'])
y = data['Class']

# Initialize cross-validation strategy
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Dictionary to store results with cross-validation
cv_results = {}

# 1. Logistic Regression with Cross-Validation
logistic_model = LogisticRegression(max_iter=1000, random_state=42)
logistic_y_pred_cv = cross_val_predict(logistic_model, X, y, cv=cv)
logistic_conf_matrix_cv = confusion_matrix(y, logistic_y_pred_cv)
```

```
logistic_accuracy_cv = accuracy_score(y, logistic_y_pred_cv)
logistic_errors_cv = logistic_conf_matrix_cv[0, 1] + logistic_conf_matrix_cv[1, 0]

cv_results['Logistic Regression'] = {
    'Confusion Matrix': logistic_conf_matrix_cv,
    'Errors': logistic_errors_cv,
    'Accuracy': logistic_accuracy_cv,
    'Predictions': logistic_y_pred_cv
}

# 2. K-Nearest Neighbors with Cross-Validation
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_y_pred_cv = cross_val_predict(knn_model, X, y, cv=cv)
knn_conf_matrix_cv = confusion_matrix(y, knn_y_pred_cv)
knn_accuracy_cv = accuracy_score(y, knn_y_pred_cv)
knn_errors_cv = knn_conf_matrix_cv[0, 1] + knn_conf_matrix_cv[1, 0]

cv_results['K-Nearest Neighbors'] = {
    'Confusion Matrix': knn_conf_matrix_cv,
    'Errors': knn_errors_cv,
    'Accuracy': knn_accuracy_cv,
    'Predictions': knn_y_pred_cv
}

# 3. Naive Bayes with Cross-Validation
nb_model = GaussianNB()
nb_y_pred_cv = cross_val_predict(nb_model, X, y, cv=cv)
nb_conf_matrix_cv = confusion_matrix(y, nb_y_pred_cv)
nb_accuracy_cv = accuracy_score(y, nb_y_pred_cv)
nb_errors_cv = nb_conf_matrix_cv[0, 1] + nb_conf_matrix_cv[1, 0]

cv_results['Naive Bayes'] = {
    'Confusion Matrix': nb_conf_matrix_cv,
    'Errors': nb_errors_cv,
    'Accuracy': nb_accuracy_cv,
    'Predictions': nb_y_pred_cv
}

# 4. Decision Tree with Cross-Validation
dt_model = DecisionTreeClassifier(random_state=42)
dt_y_pred_cv = cross_val_predict(dt_model, X, y, cv=cv)
dt_conf_matrix_cv = confusion_matrix(y, dt_y_pred_cv)
dt_accuracy_cv = accuracy_score(y, dt_y_pred_cv)
dt_errors_cv = dt_conf_matrix_cv[0, 1] + dt_conf_matrix_cv[1, 0]

cv_results['Decision Tree'] = {
```

```
    'Confusion Matrix': dt_conf_matrix_cv,
    'Errors': dt_errors_cv,
    'Accuracy': dt_accuracy_cv,
    'Predictions': dt_y_pred_cv
}

# Create a summary of results with cross-validation
cv_formatted_results = []
for model_name, metrics in cv_results.items():
    cv_formatted_results.append({
        'Model': model_name,
        'Confusion Matrix': metrics['Confusion Matrix'],
        'Errors (FP + FN)': metrics['Errors'],
        'Accuracy': metrics['Accuracy'],
        'First 5 Predictions': metrics['Predictions'][:5]
    })

# Convert to a DataFrame for better visualization
cv_results_df = pd.DataFrame(cv_formatted_results)
print(cv_results_df)
```

Cross-Validation Classification Model Results

	Model	Confusion Matrix	Errors (FP + FN)	Accuracy	First 5 Predictions
1	Logistic Regression	[[434 10] [11 228]]	21	0.9692532942898975	[2 4 2 4 2]
2	K-Nearest Neighbors	[[437 7] [8 231]]	15	0.9780380673499268	[2 4 2 4 2]
3	Naive Bayes	[[424 20] [6 233]]	26	0.9619326500732065	[2 4 2 4 2]
4	Decision Tree	[[428 16] [19 220]]	35	0.9487554904831625	[2 4 2 4 2]

After optimization, KNN is the best-fit model.

1) Try other classification models using ChatGPT.